Structure-Based Reasoning Methodology

Core Specification of a Constraint-Driven Cognitive Engine

Section 1 — Overview

본 문서는 문제 해결을 위한 기법이나 절차를 설명하는 문서가 아니다.

이는 문제를 다루는 **사고 구조 자체**를 정의하기 위한 문서이다.

본 방법론은 특정 기술, 도메인, 직무에 종속되지 않으며,

문제를 **구조·경계·패턴·수렴**의 관점에서 해석하는

범용 구조 기반 추론 엔진을 정의한다.

사례, 기술, 경험은 본 방법론의 입력값에 해당하며,

방법론 자체는 **불변 제약과 절차**로 구성된다.

Section 2 — Methodology Definition & Cognitive Architecture

2.1 Methodology Overview

본 방법론은 지식의 축적이나 경험의 양을 전제로 하지 않는다.

문제는 항상 구조적 형태로 해석되며,

입력 도메인과 무관하게 동일한 사고 파이프라인이 작동한다.

2.2 Core Structural Reasoning Loop

본 방법론의 핵심 사고 루프는 다음 순서를 따른다.

- **Boundary** — 문제의 경계를 명시적으로 정의
- **Structure** — 구조적 관계 식별
- **Pattern** — 반복성 및 충돌 구조 탐색

- **Convergence** — 구조적으로 더 이상 분기되지 않는 상태로 수렴

이 루프는 의식적 설명 이전 단계에서 작동하며,

의식은 결과를 검증·언어화하는 역할을 수행한다.

2.3 Cognitive Architecture Layers

본 방법론은 다층적 인지 구조를 가진다.

- 무의식적 구조 추출 계층
- 관찰자(Observer) 계층
- 출력 제어 계층

이 계층 분리는 자기참조적 분석과 설명 가능성을 보장한다.

2.4 Governing Constraints

본 방법론은 사고를 유도하는 규칙이 아니라,

사고 결과를 **제약·검증·정렬**하기 위한

불변 원리 집합에 의해 제한된다.

해당 원리의 전문은 **Appendix C**에 명세된다.

→ Appendix C

2.5 Responsibility-Gated Output

사고 과정 자체에는 제한을 두지 않으나,

외부 출력 및 실행은

책임 가능성 범위(Responsibility Boundary) 내에서만 허용된다.

Diagram 1 — Core Engine + Constraints + Output Gate

```
flowchart TB
  A[Boundary] ⟶ B[Structure]
  B ⟶ C[Pattern]
  C ⟶ D[Convergence]

  subgraph CORE[Core Structural Reasoning Loop]
```

```
  A
  B
  C
  D
end

E[11 Governing Constraints<br>Appendix C]
E → A
E → B
E → C
E → D

D → F[Responsibility Output Gate P10]
F → G[External Output]
```

Section 3 — Application Case Study

Structural Reasoning in Practice

3.1 Case Overview

- 사례: **MS365 Copilot Shortcut Interference Issue**
- 문제 유형: 오류 메시지 없는 예상 외 실행(Event Substitution)

본 사례는

비가시 신호 감지와 플랫폼 레벨 구조 충돌이

동시에 나타난 분석 사례이다.

3.2 Boundary Definition

문제는 다음과 같이 재정의되었다.

> “단축키 입력이 실패했는가?”가 아니라
>
> **“입력 이후 어떤 구조가 해당 이벤트를 선점했는가?”**

3.3 Execution Trace — Actual Reasoning Path

- 서드파티 애플리케이션 호출 시 Copilot 실행
- 단축키 가설 수립 및 변경 시도
- 결과 변화 없음 → **의미 없는 변화로 판단**
- Copilot 및 서드파티 앱이 모두 Electron 기반임을 확인
- Global Shortcut 충돌 가능성 의심
- uiOhook / Keypress OSD 로그를 통해 입력 선점 확인
- 구조적 원인으로 수렴

3.4 Structural Mapping

- **Structure**: Electron을 기술이 아닌 **플랫폼 계층**으로 해석
- **Pattern**: Electron 계열 간 Global Shortcut 선점
- **Convergence**: Copilot 플랫폼 레벨 입력 선점 구조

3.5 Constraint Activation

본 사례에서 활성화된 제약:

- P3 — Minimal Sufficiency
- P5 — Non-Event Sensitivity
- P6 — Pattern Legitimacy
- P7 — Convergence Validity
- P10 — Responsibility Constraint

3.6 Output Control

- 확정 출력: Copilot의 플랫폼 레벨 입력 선점
- 미확정 유지: 타 시스템 일반화 여부

본 출력은

책임 제약에 의해 의도적으로 제어되었다.

3.7 External Technical Evidence

본 사례의 상세 기술 분석과 실험 기록은

외부 GitHub 커뮤니티에 공개된 RCA 문서로 제공된다.

- https://github.com/SnosMe/awakened-poe-trade/issues/1737

Section 4 — Scope, Limits, and Structural Closure

4.1 What This Methodology Is

본 방법론은 특정 기술, 도메인, 직무에 종속되지 않는다.

이는 문제를 **구조·경계·패턴·수렴**의 관점에서 해석하기 위한

범용 구조 기반 추론 엔진이다.

4.2 What This Methodology Is Not

본 방법론은 다음을 목표로 하지 않는다.

- 특정 기술 스택이나 도구 사용 지침
- 문제 해결을 자동화하는 규칙 집합
- 경험이나 직관을 대체하는 결정 트리

본 방법론은 사고를 대신 수행하지 않으며,

사고의 **형태·순서·제약 조건**만을 정의한다.

4.3 Applicability and Limits

본 방법론은 다음 조건에서 가장 효과적으로 작동한다.

- 문제의 경계가 정의 가능한 경우
- 구조적 관계가 결과에 결정적 영향을 미치는 경우
- 결과보다 원인 구조가 중요한 경우

반대로 다음 영역에서는 제한적인 효용을 가진다.

- 순수 반복 작업
- 규칙이 완전히 고정된 문제
- 물리적 숙련이 핵심인 영역

4.4 Responsibility and Output Control

본 방법론은 사고 과정 자체에는 제한을 두지 않으나,

외부 출력은 항상

책임 가능성 범위 내에서만 허용된다.

이는 보수성이 아니라

결과의 신뢰성과 재현 가능성을 위한 설계이다.

4.5 Node-Based Instantiation and Emergent Expansion

미시적 관점에서 본 방법론은

원리와 절차가 고정된 상태에서

독립적으로 완결된 분석 노드(Node)만을 생성한다.

각 노드는 서로를 수정하거나 보강하지 않으며

단독으로 검증·참조·폐기 가능하다.

그러나 거시적 관점에서

이러한 노드의 누적은

적용 가능한 문제 공간의

자연스러운 확장(Emergent Expansion)으로 관찰될 수 있다.

즉, 본 방법론은

내부적으로는 불변이며,
외부적으로는 성장하는 구조를 가진다.

Diagram 2 — Node-Based Instantiation + Emergent Expansion

```
flowchart LR
  CORE[Core Engine]

  A[Case Node A] ⟶ CORE
  B[Case Node B] ⟶ CORE
```

```
C[Case Node C] ⟶ CORE
D[External Node<br>GitHub RCA] ⟶ CORE
```

4.6 Interpretation and Usage Guidance

본 문서는 특정 문제에 대한 정답을 제시하지 않는다.

이는 문제를 다루는 **사고 구조와 제약 체계**를 정의한다.

적용 결과는 항상

해당 사례 노드의 맥락 내에서만 해석되어야 하며,

일반화는 중앙 엔진 수준에서만 정당화될 수 있다.

4.7 Closing Statement

본 방법론은 완결된 규칙 집합이 아니라,

일관된 사고를 가능하게 하는

구조적 제약 시스템이다.

문서에 포함된 사례와 외부 실증은

방법론의 일부가 아니라,

동일한 엔진에 의해 생성된 **개별 노드**에 해당한다.

본 문서는 이 구조를 정의하고 고정하는 것을 목표로 하며,

추가 설명이나 확장은 요구하지 않는다.

Note:

본 문서는 구조 기반 추론 **코어 엔진의 명세(Core Specification)**를 정의한다.

엔진의 호출 조건, 다른 사고 방식과의 공존, 운용 및 수명 주기에 관한 내용은

본 문서의 범위를 벗어나며, 필요 시 별도 문서로 분리되어 다뤄질 수 있다.

Appendix A — Academic Background & Origin

(Origin Anchor)

- 2007–2011 독자적 바이너리 분석 및 역공학 연구
- 정규 교육 이전 형성된 분석 역량
- 교육은 이론적 정렬 단계로 작동

→ Appendix A

Appendix C — Constraint Specification

(11 Governing Principles)

- Structural Primacy
- Boundary Explicitness
- Minimal Sufficiency
- Invariance over Instance
- Non-Event Sensitivity
- Pattern Legitimacy
- Convergence Validity
- Cross-Domain Consilience
- Explanation Separability
- Responsibility Constraint
- Self-Consistency

→ Appendix C

Diagram 3 — Reasoning vs Output Separation

```
flowchart TB
  R[Internal Reasoning Space] ⟶ K[Conclusion Candidates]
  K ⟶ O[External Output]
  K ⟶ Q[Deferred or Withheld]
```

Appendix D — Behavioral Proof

(Responsibility Constraint)

- Copilot RCA에서의 의도적 미확정 결정
- 사고와 출력의 분리 실증

→ Appendix D

📌 Document Status

- Version: **v1.0 Final**
- Core Engine: **Locked**
- Expansion Policy: **Node Generation Only**
- External Evidence: **GitHub-based**
- Output Format: **Notion (Primary)**

🔑 Final Statement

> 이 문서는 문제를 푸는 방법을 설명하지 않는다.
> 문제를 다루는 사고 구조를 정의한다.